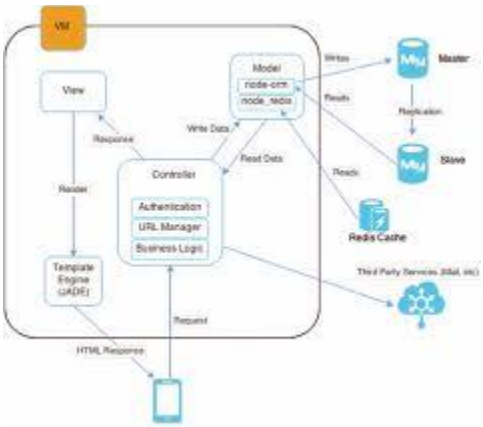
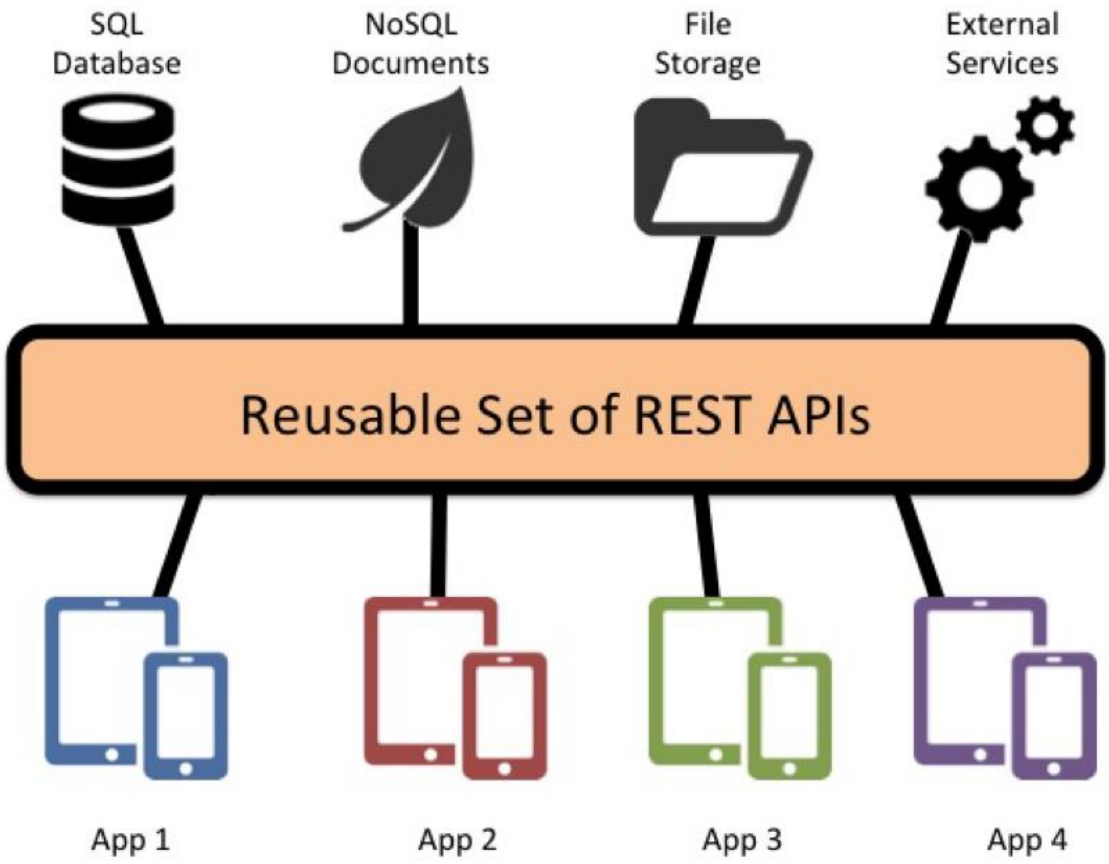
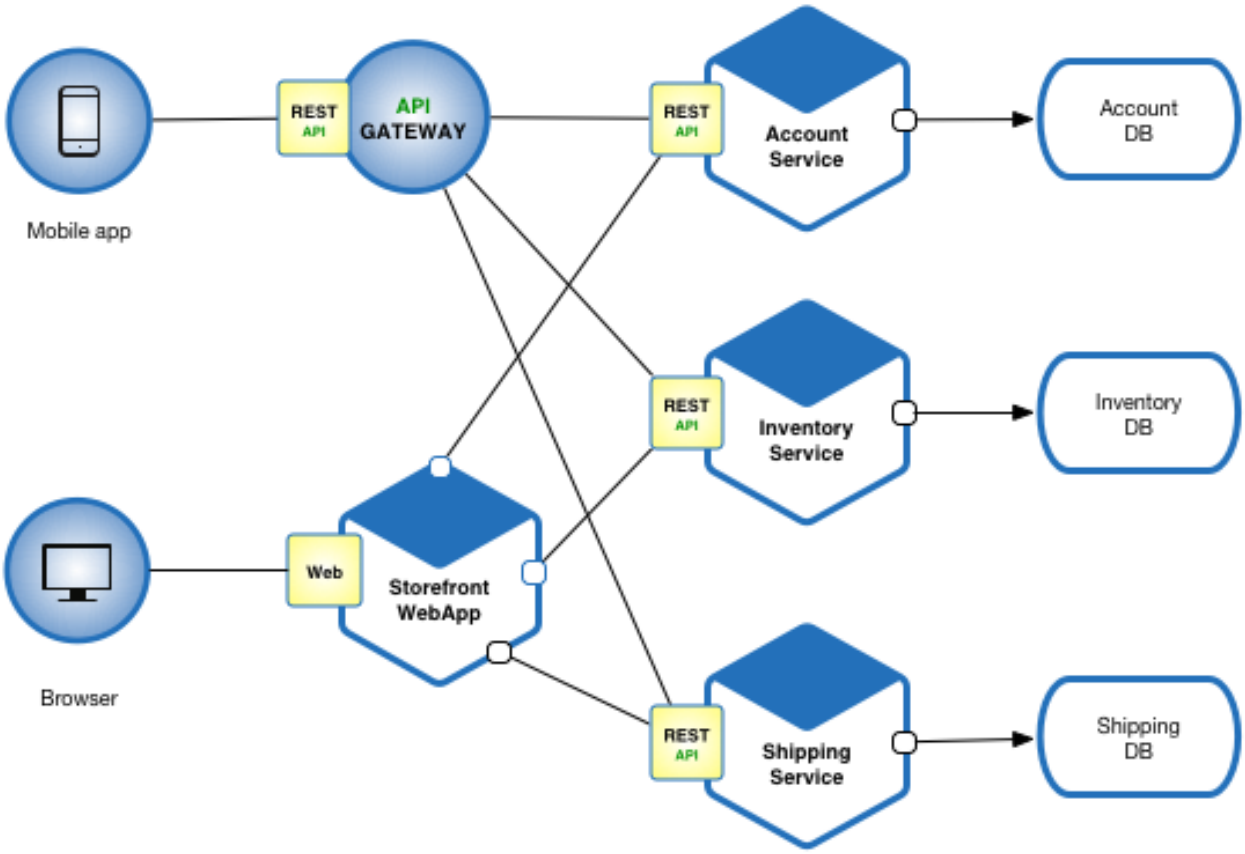
Backend Frameworks & Application Architecture







Purpose of Backend Frameworks & Architecture

Backend frameworks and application architecture define **how software works behind the scenes**.
They control **business logic, data flow, security, performance, and scalability**.

At **ENGLORAY Tech Group**, this resource ensures developers:

- Build clean and maintainable systems
- Handle complex logic efficiently
- Scale applications as usage grows
- Collaborate smoothly across teams

1. Backend Frameworks (Execution Layer)

Frameworks speed up development by providing **ready-made structures and best practices**.

Node.js + Express

Used For

- REST API development
- Real-time applications
- Microservices

Why ENGLORAY Uses It

✓ Fast and scalable
✓ JavaScript end-to-end (frontend + backend)
✓ Large ecosystem

Django

Used For

- Secure backend systems
- Admin dashboards
- Data-driven applications

Why It's Important

✓ Built-in security
✓ Rapid development
✓ Clean MVC structure

Spring Boot

Used For

- Enterprise-grade applications
- Large-scale systems
- Microservices

Why ENGLORAY Supports It

✓ Strong architecture
✓ High performance
✓ Long-term stability

ASP.NET Core

Used For

- High-performance APIs
- Enterprise software
- Secure backend services

Why It Matters

✓ Excellent performance
✓ Strong Microsoft ecosystem
✓ Clean layered architecture

2. Application Architecture Patterns (System Design)

Architecture defines **how components are structured and communicate**.

◇ Monolithic Architecture

- Single codebase
- Simple structure
- Easy to start

Used When

✓ Small to medium projects

◇ Layered Architecture

- Presentation layer
- Business logic layer
- Data access layer

Why ENGLORAY Uses This

✓ Clean separation of concerns

✓ Easier maintenance

◇ Microservices Architecture

- Independent services
- API-based communication
- Scalable systems

Used When

✓ Large or growing applications

◇ MVC / MVVM Patterns

- Clear responsibility separation
- Organized code structure

Benefit

✓ Better readability

✓ Team-friendly development

3. API Design & Communication

APIs act as the **bridge between frontend, backend, and external services**.

Standards Followed

- RESTful APIs
- JSON-based responses
- Proper status codes

Tools Used

- **Postman**

Why This Matters

✓ Smooth frontend-backend integration
✓ Reusable services
✓ Mobile & third-party support

4. Security Architecture

Security is built **into the architecture**, not added later.

Practices Used

- Authentication & authorization
- Token-based security (JWT)
- Role-based access control
- Secure API gateways

Why ENGLORAY Enforces This

✓ Protects data
✓ Prevents breaches
✓ Builds client trust

5. Performance, Scalability & Reliability

Architecture is designed to **handle growth and load**.

Techniques Used

- Caching
- Load balancing
- Database optimization
- Async processing

Benefit

✓ Faster responses
✓ High availability
✓ Scalable systems

6. Documentation & Maintainability

Good architecture is **well-documented**.

ENGLORAY Practices

- API documentation
- Code comments
- Architecture diagrams

Why This Matters

✓ Easy onboarding
✓ Smooth maintenance
✓ Long-term clarity

Final Outcome of Backend Frameworks & Application Architecture

With strong frameworks and architecture, ENGLORAY software developers can:

✓ Build secure and scalable systems
✓ Manage complex business logic
✓ Maintain clean and organized code
✓ Scale applications confidently
✓ Deliver enterprise-ready solutions